Developer Guide — Knotted Coil Designer

This guide is for anyone extending the tool. Read it together with:

- `Design.md`, the frozen specification and its changelog;
- `DECISIONS.md`, why things are the way they are;
- `docs/REQUEST_LOG.md`, what was asked for and when.

1. What the tool does, in one paragraph

It designs MRI receive phased arrays made of **flat knotted wire loops** (torus knots T(p,q) drawn as rosettes) on a helmet. The pieces are:

- **Element shape:** knot type, bloom (lobe length), petal sharpness (lobe narrowness), rotation, wire.
- **Electromagnetics:** a quasi-static thin-wire model. It gives mutual and self-inductance (Neumann formula), the coupling coefficient k, and the B1 field (Biot–Savart).
- **Solver:** arranges N elements on the helmet so neighbouring elements are geometrically decoupled (k ≈ 0) while covering the region.
- **Front end:** a single browser page with no build step, plus Node.js tests.

2. Repository map

Path	What it is
`core/knot.js`	Element geometry: `points`, crossings and over/under pattern, lift solver, clearance, bend radius, lobe aspect ratio, bloom floor, adaptive point count
`core/inductance.js`	Neumann mutual inductance (with near-field refinement), self-inductance, Maxwell coaxial closed form, elliptic integrals
`core/surface.js`	Helmet ellipsoid, coverage region, local frames, wrapping a flat element onto the surface
`core/layout.js`	Seeded random numbers; initial even spread of N centres (farthest-point sampling + Lloyd)
`core/optimise.js`	Decoupling solver: stage 1 lookup table, stage 2 exact; pair scan; final report
`core/field.js`	Biot–Savart B1; head surface and slice sampling
`app/index.html`	The whole UI: Element, Pair explorer, Array on helmet, exports. Heavy maths runs in Web Workers
`tests/run.js`	Validation checks V1–V8 (pre-registered) plus labelled post-hoc checks. Writes `validation/`
`tests/compare_worstpair.js`	Pre-registered paired solver comparison (protocol in `validation/`)
`tools/serve.js`	Tiny local web server: `node tools/serve.js` → http://localhost:8765/app/index.html
`tools/build_dist.js`	Builds `dist/`, a flat copy of the app for hosting or sending
`vendor/`	Three.js 0.149.0 (MIT), pinned and stored locally so the app works offline
`docs/`	Request log; copies of the research-integrity rules this project follows
`sources/`	Archived primary sources for VERIFIED citations

3. How the code is organised

- **Plain JavaScript, no build step, no dependencies** apart from the vendored Three.js.
- **Module pattern:** every `core/*.js` file wraps itself in a factory that
  - attaches to a global `KC` namespace (`KC.knot`, `KC.ind`, `KC.surface`, `KC.layout`, `KC.opt`, `KC.field`);
  - pushes the factory onto `KC_FACTORIES`;
  - exports `KC` for Node.
- **Same code in Node and the browser.** The app builds its Web Worker by concatenating `KC_FACTORIES[i].toString()` into a Blob, so the worker runs exactly the same code. **Load order matters:** knot, inductance, surface, layout, optimise, field.
- **Units:** mm, nH, µT/A, radians internally (degrees only in the UI).

    - Coordinates: x is left–right, y is back–front, z is up.
- **The element formula** (Design §3.1, v1.4):
    - $\psi = q\phi + a\cdot\sin(q\phi)$, with $a = 1 - 1/m$ (sharpness m);
    - $r = R(1 + \beta(1 + \cos\psi)/2)$;
    - $x = r\cdot\cos(p\phi + \theta)$, $y = r\cdot\sin(p\phi + \theta)$, $z = h\cdot\sin\psi$.
    - The v1.0 "symmetric" shape ($r = R(1 + \beta\cos q\phi)$) is still selectable.

4. Common extensions — where to start

You want to…	Do this
Add a knot family (Lissajous, Fourier)	Add a generator next to `knot.points` that returns the same `[[x,y,z],…]` format, with a z-lift that encodes over/under. Everything downstream (crossings, lift, inductance, solver, B1) works on point lists. Add a V-check: crossing count and pattern equal the family's known diagram
Use a real helmet former	Replace the ellipsoid in `core/surface.js`: `normal`, `snap`, `alongNormal`, `inCoverage`, `samples`, `coveredArea`, `point`. An STL importer would need nearest-point projection and normals
Mix element types in one array	`optimise.js` assumes one shared element shape (one lookup table, one self-inductance). Store per-element shape parameters; build one table per distinct pair of shapes
Improve the solver	It is a transparent accept-if-better local search and is **strongly seed-sensitive** (DECISIONS.md). Cheapest gain: multi-seed restarts, keeping the best. Any change → pre-register a paired comparison like `validation/COMPARE_worstpair_PROTOCOL.md`
Better field maps	Export wire paths (CSV, OBJ/STL) to a full-wave solver: openEMS (free), CST, HFSS, Sim4Life. Don't add E-field maps to this quasi-static model (DECISIONS.md explains why)
Add an export	`app/index.html`, "exports" section. `arrRes` holds the solved array (`wires`, `centres`, `theta`, `kMatrix`, `pairs`, `config`)

5. Rules this project follows (please keep them)

These come from the research-integrity framework in `docs/research_integrity/`:

- **Freeze, then change by changelog.** `Design.md` is frozen. Any change to geometry, the EM model or validation criteria gets a version bump and a dated changelog entry saying who asked, why, and what it affects.
- **Pre-register checks.** Write the pass criterion into `Design.md` §7 (or a protocol file) and **commit before running**. Report failures as failures: the git history keeps every failing run, e.g. runs 1, 4 and 5. Never edit a criterion after seeing results. Fixes to the test harness are logged as such.
- **Tag every parameter's origin** (LIT / CONV / JUDG / TUNED / DERIVED, with PLACEHOLDER where relevant), inline in the code *and* in `PARAMETERS.md`.
- **Cite only VERIFIED sources.** Read the primary text and archive it in `sources/`. Label anything else UNVERIFIED.
- **Provenance header** on every source file (AI-assisted authorship, accountable person).

- **Label results exploratory** unless validated to the tier the claim needs. Every coupling and B1 output says so.
- **Decision log:** dated entries in `DECISIONS.md`, including mistakes and how they were found.

6. Run, test, build

```
node tools/serve.js
```

Then open http://localhost:8765/app/index.html. Double-clicking `app/index.html` may also work, but this hasn't been tested.

```
node tests/run.js
```

Validation suite, about 5–15 minutes. Writes `validation/validation_report.{html,json}` and `validation_status.js`, which drives the app's header banner.

```
node tests/compare_worstpair.js
```

Re-runs the pre-registered worst-pair comparison, about 3–5 minutes.

```
node tools/build_dist.js
```

Builds `dist/`, a flat copy of the app for hosting.

7. Known limitations and open items (as of 2026-10-20)

- **Quasi-static thin-wire model only.** No capacitors, tissue, wave effects or preamplifier decoupling.
- **Placeholders:** wire spec and helmet dimensions (PARAMETERS.md).
- **V3:** cannot be evaluated until Roemer et al. 1990 is read (paywalled).
- **V6:** fails below the wire-dependent bloom floor. It is a known physical limit, shown in the UI.
- **Solver:** seed-sensitive. No 32-element test array met $|k| < 0.01$ for all nearest pairs.
- **Very sharp petals** (high m, 3–5 windings) need crossing lifts of several mm.
- **Untested:** opening the app from `file://` (double-click). The local server is tested.